

Concepts & Study Guide

Restaurant Table Reservation & Waitlist System

Java Programming - Case Study 70 | Aditya Sunil Chouksey (150096725070)

How to use this guide

Every concept used in the project has its own chapter: what it is in plain words, exactly where and how this project uses it (the code shown is copied automatically from the real source files), a worked example you can trace on paper, the traps examiners like, and likely questions with short answers.

Chapters marked **■ Required by the brief** are the fourteen concepts the case study asks for. Chapters marked **□ Added by us** were introduced to make the system work well - be ready to explain why you added them.

Suggested order: read chapters 6 (Collections), 12 (Threads) and 16 (Algorithms) first - they carry the most marks and the most questions.

Concept map

#	Concept	Brief?	Where in the code
1	Classes & Objects	Required	model/*.java
2	Constructors	Required	Table, Reservation, WaitlistEntry, RestaurantManager
3	Encapsulation, static, final, access modifiers	Added	every class
4	Enum	Required	TableStatus, ReservationStatus, Section, SortField
5	Inheritance, interfaces, Observer pattern	Added	exceptions, panels, Runnable, RestaurantListener
6	Collections: ArrayList, LinkedList, HashMap, TreeMap	Required	RestaurantManager, Waitlist
7	CRUD	Required	RestaurantManager reservation methods
8	Searching	Required	searchReservations, HashMap lookup
9	Sorting, Comparator, lambdas	Required	SortField, RestaurantManager.sort
10	Exception handling	Required	exception/*, every GUI action
11	Validation, regular expressions	Required	Validator, TimeUtil
12	Runnable/Thread (+ synchronized, volatile, EDT)	Required (+ added)	WaitlistNotifier, ReservationMonitor, MainFrame

#	Concept	Brief?	Where in the code
13	Swing GUI	Required	gui/*.java
14	java.time and Clock	Added	Reservation, TimeUtil, RestaurantManager
15	Serialization and file I/O	Added	persistence/*
16	Algorithms: best fit, interval overlap, fair queue	Design	RestaurantManager, Waitlist
17	Packages, compiling and running	Added	run.sh, javac/java

1. Classes & Objects

■ Required by the brief

In one sentence

A class is a blueprint that bundles data (fields) with the operations on that data (methods); an object is one concrete thing built from that blueprint, with its own copy of the fields.

What it is

Think of *Table* as the form a restaurant would print for every table: number, seats, section, status. The class is the blank form; each filled-in form - table 1, table 2, table 12 - is an object. Every object has its own field values but shares the same methods.

How this project uses it

Class	Represents	Key fields
Table	A physical table	number, capacity, section, status, heldFor, occupiedBy
Reservation	A booking (or a seated walk-in)	id, customer, partySize, dateTime, tableNumber, status, walkIn
Customer	A guest	name, phone
WaitlistEntry	A walk-in party waiting	id, customer, partySize, preferredSection, joinedAt, heldTableNumber
Waitlist	The queue of waiting parties	queue (LinkedList<WaitlistEntry>)

The methods that change a table's state live inside the Table class itself, so a table can never be "occupied but also held for someone":

model/Table.java

```
public void hold(String id) {
    status = TableStatus.RESERVED;
    heldFor = id;
    occupiedBy = null;
}
```

(hold, occupy and free each set status and the two id fields together.)

Worked example

1. `Table t = new Table(5, 6, Section.MAIN_HALL);` creates one object: number 5, 6 seats, status AVAILABLE.
2. `t.hold("W002");` - Karan Malhotra's party is next: status becomes RESERVED, heldFor = "W002".
3. `t.occupy("WI1013");` - they sit down: status OCCUPIED, occupiedBy = "WI1013", heldFor cleared.
4. `t.free();` - they leave: back to AVAILABLE with both ids cleared.

Viva traps - say this, not that

A class is **not** an object. "Table" is the class; "table 5" is an object (an instance).

Two Customer objects with the same phone are *equal* (equals() compares phone) but are still two different objects in memory (== is false).

Likely questions

Question	Short answer
Why is Reservation a class and not just a row of strings?	It groups related data with behaviour (getEndTime()) and type safety - partySize is an int, dateTime a LocalDateTime, status an enum.
How many Table objects exist in the sample data?	Twelve - one per physical table.
Why does Customer override equals() and hashCode()?	So two Customer objects with the same phone number count as the same guest - used for duplicate-booking checks.

2. Constructors

■ Required by the brief

In one sentence

A constructor is the special method that runs once when an object is created with new, and puts the object into a valid starting state.

How this project uses it

A new table always starts AVAILABLE - the constructor guarantees it, so no caller can forget:

model/Table.java

```
public Table(int number, int capacity, Section section) {
    this.number = number;
    this.capacity = capacity;
    this.section = section;
    this.status = TableStatus.AVAILABLE;
}
```

A reservation generates its own unique ID from a shared counter and always starts CONFIRMED:

model/Reservation.java

```
public Reservation(Customer customer, int partySize, LocalDateTime dateTime,
    Section preferredSection, String notes, boolean walkIn) {
    this.id = (walkIn ? "WI" : "R") + SEQUENCE.incrementAndGet();
    this.customer = customer;
    this.partySize = partySize;
    this.dateTime = dateTime;
    this.preferredSection = preferredSection;
    this.notes = notes == null ? "" : notes.trim();
    this.walkIn = walkIn;
    this.status = ReservationStatus.CONFIRMED;
    this.createdAt = LocalDateTime.now();
}
```

The SEQUENCE counter is static (one per class, shared by all objects) and an AtomicInteger so two threads creating bookings at once can never get the same number. Walk-ins get the prefix "WI", bookings "R".

RestaurantManager shows constructor chaining - the no-argument constructor calls the other one with this(...):

service/RestaurantManager.java

```
public RestaurantManager() {
    this(Clock.systemDefaultZone());
}
```

service/RestaurantManager.java

```
/** A custom clock lets tests run against a fixed "now". */
public RestaurantManager(Clock clock) {
    this.clock = clock;
}
```

Worked example

The first booking made in a run gets `SEQUENCE.incrementAndGet()` = 1001, so its id is "R1001"; the next walk-in gets 1002 -> "W1002". After loading saved data, `ensureSequenceAtLeast()` moves the counter past the highest saved id so new ids never clash with old ones.

Viva traps - say this, not that

A constructor has no return type - not even void. If you write "public void Table(...)" it becomes an ordinary method and the default constructor is used instead.

If a class defines any constructor, Java no longer provides the free no-argument one.

Likely questions

Question	Short answer
What does this(clock) do?	Calls another constructor of the same class; it must be the first statement. It avoids duplicating initialisation code.
Why is the ID generated in the constructor and not passed in?	So every reservation gets a unique id automatically; callers cannot forget or duplicate one.
Why AtomicInteger instead of a static int?	id++ on an int is three steps (read, add, write); two threads could read the same value. incrementAndGet() is one atomic step.

3. Encapsulation, static, final and access modifiers

❑ Added by us - not in the brief

In one sentence

Encapsulation means hiding an object's fields behind methods, so the object controls how its own data changes.

How this project uses it

- All fields are `private`. Other classes read them through getters and change them only through meaningful methods (`hold()`, `occupy()`, `setStatus()`).
- `RestaurantManager` never hands out its real lists: `getTables()` returns `new ArrayList<>(tables)` - a copy - so the GUI cannot add or remove tables behind the manager's back.
- `final` fields (`private final int number` in `Table`) can be set once in the constructor and never changed - a table's number is its identity.
- `static final` constants such as `Reservation.DINING_MINUTES` and `Validator.MAX_PARTY_SIZE` keep business rules in one place.
- GUI panel classes have *package-private* access (no modifier): only `MainFrame` in the same package needs them.

Modifier	Visible from
<code>private</code>	Only the same class
(none) package-private	Classes in the same package
<code>protected</code>	Same package + subclasses
<code>public</code>	Everywhere

Likely questions

Question	Short answer
Why return a copy of the list instead of the list itself?	If the GUI changed the real list (say, removed a table) the <code>HashMap/TreeMap</code> and holds would no longer match - returning a copy protects the manager's invariants.
What is the difference between <code>static</code> and <code>final</code> ?	<code>static</code> = belongs to the class, one copy shared by all objects. <code>final</code> = cannot be reassigned after it is set. <code>static final</code> together = a constant.

4. Enum

■ Required by the brief

In one sentence

An enum is a type with a fixed, named set of values - the compiler rejects anything else.

How this project uses it

The brief asks for exactly this enum. Each constant also carries a readable label for the GUI:

model/TableStatus.java

```
/** Live front-of-house state of a physical table. */
public enum TableStatus {
    AVAILABLE("Available"),
    RESERVED("Reserved"),
    OCCUPIED("Occupied");

    private final String label;

    TableStatus(String label) {
        this.label = label;
    }

    public String getLabel() {
        return label;
    }

    @Override
    public String toString() {
        return label;
    }
}
```

Enums can have methods. ReservationStatus knows which statuses still block a table:

model/ReservationStatus.java

```
/** Active reservations still block a table; closed ones do not. */
public boolean isActive() {
    return this == CONFIRMED || this == SEATED;
}
```

The most advanced use is `SortField` in `RestaurantManager`: each constant stores its own `Comparator`, so "sort by party size" is just `SortField.PARTY_SIZE.comparator()` - no if/else chain (see chapter 9).

Enum	Values
TableStatus	AVAILABLE, RESERVED, OCCUPIED
ReservationStatus	CONFIRMED, SEATED, COMPLETED, CANCELLED, NO_SHOW
Section	MAIN_HALL, PATIO, BAR, PRIVATE_ROOM
NotificationType	INFO, WARNING, ALERT
SortField / SearchField	TIME, PARTY_SIZE, CUSTOMER, TABLE, STATUS / CUSTOMER, TABLE, ID

Worked example - why not use Strings?

With a String, `table.status = "Ocupied"` (typo) compiles and silently breaks every check. With the enum, `TableStatus.OCUPIED` does not compile. Enums can also be used in switch statements - `StatusCellRenderer` switches on `TableStatus` to choose the cell colour - and as keys of an `EnumMap` (`ReportGenerator` counts tables per status with one).

Viva traps - say this, not that

Compare enums with `==` (it is safe and null-safe on the left), e.g. `t.getStatus() == TableStatus.AVAILABLE`. `equals()` also works but is not needed.

Likely questions

Question	Short answer
Can an enum have a constructor?	Yes - always private (implicitly). <code>TableStatus(String label)</code> stores the label for each constant.
Where are the three table statuses changed?	Only in <code>Table.hold()</code> , <code>occupy()</code> and <code>free()</code> , called by <code>RestaurantManager</code> .
Why is <code>RESERVED</code> used for both bookings and waitlist?	In both cases the table is being kept for a specific party; <code>heldFor</code> says which one (an <code>R...</code> id or a <code>W...</code> id).

5. Inheritance, interfaces and the Observer pattern

❏ Added by us - not in the brief

In one sentence

Inheritance lets a class reuse and extend another class (extends); an interface is a contract of methods that unrelated classes can promise to provide (implements).

Inheritance in this project

- Exceptions: `RestaurantException` extends `Exception`, and four specific exceptions extend `RestaurantException` (chapter 10).
- GUI: `MainFrame` extends `JFrame`; the four panels extend `JPanel`; `StatusCellRenderer` extends `DefaultTableCellRenderer` and overrides one method to colour status cells.

Interfaces in this project

- `Runnable` (from Java) - `WaitlistNotifier` and `ReservationMonitor` implement `run()` so a `Thread` can execute them (chapter 12).
- `RestaurantListener` (ours) - the Observer pattern:

service/RestaurantListener.java

```
/**
 * Observer for {@link RestaurantManager} events. Callbacks may arrive on background threads,
 * so GUI implementations must hand work over to the Swing event thread.
 */
public interface RestaurantListener {
    void onNotification(NotificationType type, String message);

    void onDataChanged();
}
```

The Observer pattern

`RestaurantManager` (the "subject") keeps a list of listeners and calls them when something happens. `MainFrame` (the "observer") implements `RestaurantListener` and refreshes the screen. The manager never imports any Swing class, so the same manager works with the GUI, with background threads and in testing.

gui/MainFrame.java

```
@Override
public void onDataChanged() {
    SwingUtilities.invokeLater(() -> {
        autosaveTimer.restart();
        // Coalesce bursts of change events into one refresh.
        if (refreshQueued) return;
        refreshQueued = true;
        SwingUtilities.invokeLater(() -> {
            refreshQueued = false;
            refreshAll();
        });
    });
}
```

Viva traps - say this, not that

extends = "is a" (MainFrame is a JFrame). implements = "can do" (WaitlistNotifier can run).
Java allows one superclass but many interfaces.

Likely questions

Question	Short answer
Why an interface for notifications instead of calling MainFrame directly?	Loose coupling: the service layer does not depend on the GUI. Any number of listeners can subscribe, and the GUI can be replaced without touching business logic.
What is method overriding? Give an example.	A subclass replaces an inherited method with its own version, e.g. StatusCellRenderer.getTableCellRendererComponent() adds colours, and toString() is overridden in Table, Customer, Reservation and WaitlistEntry.

6. Collections: ArrayList, LinkedList, HashMap, TreeMap

■ Required by the brief

In one sentence

The Java Collections Framework provides ready-made data structures; choosing the right one for each job is what makes the program simple and fast.

The four collections and why each was chosen

service/RestaurantManager.java

```
private final List<Table> tables = new ArrayList<>();
private final List<Reservation> reservations = new ArrayList<>();
private final Map<String, Reservation> reservationById = new HashMap<>();
private final TreeMap<LocalDateTime, List<Reservation>> reservationsByTime = new TreeMap<>();
private final Waitlist waitlist = new Waitlist();
```

Operation	ArrayList	LinkedList	HashMap	TreeMap
Add	O(1) at end	O(1) at either end	O(1)	O(log n)
Find by key / index	O(1) by index, O(n) by value	O(n)	O(1) by key	O(log n) by key
Remove	O(n)	O(1) with iterator	O(1)	O(log n)
Keeps order?	Insertion order	Insertion order	No order	Sorted by key
Used here for	tables, reservations	waitlist queue	reservation by ID	reservations by time

6.1 ArrayList

A resizable array. Good for "keep everything and loop over it". Tables are kept sorted by number with Collections.sort, so the best-fit loop sees them in a predictable order.

6.2 LinkedList - the waitlist

A chain of nodes, each pointing to the next and previous. Adding at the end and removing a node you are already at are O(1) - exactly what a queue of waiting guests needs. It also implements the Deque interface (addLast, peekFirst, pollFirst).

model/Waitlist.java

```
public void enqueue(WaitlistEntry entry) {
    queue.addLast(entry);
}
```

model/Waitlist.java

```
public boolean remove(String id) {
    Iterator<WaitlistEntry> it = queue.iterator();
    while (it.hasNext()) {
        if (it.next().getId().equalsIgnoreCase(id)) {
            it.remove();
            return true;
        }
    }
    return false;
}
```

Removal uses an `Iterator` - calling `list.remove()` inside a for-each loop would throw `ConcurrentModificationException`.

Worked example - who gets table 8?

Queue position	Party	Size	Wants	Fits table 8 (4 seats, Patio)?
1	W001 Isha Verma	2	Patio	Yes -> table held for W001, stop
2	W002 Karan Malhotra	5	Any	(not checked)

If instead table 2 (2 seats, Main Hall) frees: W001 wants the Patio -> skip; W002 has 5 people -> too big, skip; nobody fits, so the table simply becomes available.

6.3 HashMap - reservation by ID

A hash table: the key's `hashCode()` picks a bucket, so `get("R1003")` goes straight to the right entry - $O(1)$ on average - instead of looping over every reservation. Used for check-in, edit, cancel, delete and search by ID.

service/RestaurantManager.java

```
private void index(Reservation r) {
    reservations.add(r);
    reservationById.put(r.getId(), r);
    indexTime(r);
}
```

6.4 TreeMap - reservations in time order

A red-black tree that keeps its keys sorted. The key is the start time; the value is a `List` because two bookings can start at the same minute. Its range views answer time questions directly:

service/RestaurantManager.java

```
/** Reservations starting in [from, to), using the TreeMap's range view. */
public synchronized List<Reservation> getReservationsBetween(LocalDateTime from, LocalDateTime to) {
    List<Reservation> out = new ArrayList<>();
    for (List<Reservation> slot : reservationsByTime.subMap(from, true, to, false).values()) {
        out.addAll(slot);
    }
    return out;
}
```

Worked example

1. Bookings are added at 19:30, 13:00, 20:00 and 19:30 again (two parties).
2. The `TreeMap` keys are kept as 13:00 -> 19:30 -> 20:00 (sorted), and 19:30 maps to a list of two reservations.
3. At 19:05 the monitor asks `subMap(18:35, 19:35)`: the answer is just the two 19:30 bookings - their tables get held.
4. `tailMap(now - 30 min)` walks forward in time to find each table's next booking for the "Next booking" column.

Viva traps - say this, not that

HashMap does not keep any order - never rely on its iteration order. That is why a separate TreeMap is used for time order.

LinkedList get(i) is $O(n)$ - it is used as a queue (ends + iterator), never by index.

The three indexes must stay consistent: every add/remove/time-change updates the ArrayList, HashMap and TreeMap together (index(), unindexTime()).

Likely questions

Question	Short answer
Why three structures for reservations instead of one?	Each answers a different question quickly: the ArrayList keeps everything, the HashMap finds by ID in $O(1)$, the TreeMap gives time order and ranges.
Why is the TreeMap value a List?	Two reservations can start at exactly the same time; a map key can only appear once.
Why not an ArrayList for the waitlist?	Removing from the front or middle of an ArrayList shifts every element ($O(n)$); a LinkedList unlinks one node.
What happens to the HashMap when a reservation is deleted?	reservationById.remove(id) is called together with the list removal and unindexTime(), so no structure keeps a stale entry.

7. CRUD - reservation management

■ Required by the brief

In one sentence

CRUD is the four basic operations on stored records: Create, Read, Update, Delete.

Operation	Method in RestaurantManager	GUI button
Create	createReservation(name, phone, party, dateTime, section, table, notes)	Book
Read	getReservation(id), getReservations(), searchReservations(...)	list, search box
Update	updateReservation(id, ...), plus status changes checkIn, cancelReservation, markNoShow	Save Changes, Check In, Cancel, Mark No-show
Delete	deleteReservation(id)	Delete Record

Create, step by step

service/RestaurantManager.java

```
public synchronized Reservation createReservation(String name, String phone, int partySize,
LocalDateTime dateTime,
                                                    Section preferredSection, Integer
requestedTable, String notes)
    throws RestaurantException {
    Validator.validateCustomer(name, phone);
    Validator.validatePartySize(partySize);
    Validator.validateReservationTime(dateTime, now());
    Customer customer = new Customer(name.trim(), Validator.normalizePhone(phone));
    ensureNoOverlappingBooking(customer, dateTime, null);

    Table table = chooseTable(partySize, dateTime, preferredSection, requestedTable, null);
    Reservation r = new Reservation(customer, partySize, dateTime, preferredSection, notes,
false);
    r.setTableNumber(table.getNumber());
    index(r);

    fire(NotificationType.INFO, "Booked " + describe(r) + sectionNote(preferredSection, table));
    holdIfDue(r);
    fireDataChanged();
    return r;
}
```

1. Validate the input (throws InvalidReservationException on bad data).
2. Refuse a second overlapping booking for the same phone number.
3. Choose a table (the requested one if valid, otherwise best fit - chapter 16).
4. Create the object and index it in the ArrayList, HashMap and TreeMap.
5. Notify listeners, and hold the table immediately if the booking starts within 30 minutes.

Cancel versus Delete

Cancel keeps the record (status CANCELLED) so it still appears in reports; Delete removes it from all three structures. A seated party cannot be deleted - the table must

be released first - so the floor plan can never point at a reservation that no longer exists.

Likely questions

Question	Short answer
What happens if you edit a booking to a time when its table is taken?	updateReservation keeps the current table if it is still free; otherwise it runs the same table choice as a new booking, and throws TableNotAvailableException if nothing fits.
Why does updateReservation re-index the TreeMap?	The key is the start time; if the time changes the entry must move to the new key, otherwise time queries would miss it.

8. Searching

■ Required by the brief

In one sentence

Searching means finding the records that match a condition - here by customer, by table or by reservation ID.

service/RestaurantManager.java

```
public synchronized List<Reservation> searchReservations(String query, SearchField field) {
    String q = query == null ? "" : query.trim();
    if (q.isEmpty()) {
        return getReservations();
    }
    List<Reservation> out = new ArrayList<>();
    switch (field) {
        case ID: {
            Reservation r = reservationById.get(q.toUpperCase());
            if (r != null) out.add(r);
            break;
        }
        case TABLE: {
            int number;
            try {
                number = Integer.parseInt(q);
            } catch (NumberFormatException e) {
                return out;
            }
            for (Reservation r : reservations) {
                if (isOnTable(r, number)) out.add(r);
            }
            break;
        }
        case CUSTOMER:
        default: {
            String lower = q.toLowerCase();
            String digits = Validator.normalizePhone(q);
            for (Reservation r : reservations) {
                Customer c = r.getCustomer();
                if (c.getName().toLowerCase().contains(lower)
                    || (!digits.isEmpty() && c.getPhone().contains(digits))) {
                    out.add(r);
                }
            }
            break;
        }
    }
    return out;
}
```

Search by	Technique	Cost
Reservation ID	HashMap lookup (key normalised to upper case)	O(1)
Table number	Parse the number, then linear scan	O(n)
Customer	Linear scan; case-insensitive contains() on the name, digits-only contains() on the phone	O(n)

Worked example

Typing "YA" in the search box: the query is lower-cased to "ya"; Priya Nair, Kavya Iyer, Ananya Gupta and Diya Reddy contain "ya" and are returned; the GUI then sorts them. Typing "9876" matches every phone that contains those digits. A table search for "abc" returns nothing instead of crashing, because the NumberFormatException is caught.

Likely questions

Question	Short answer
Is this linear or binary search? Why?	Linear (plus a HashMap lookup for IDs). The list is small and "contains" matching cannot use binary search, which needs exact keys in sorted order.
How is the search case-insensitive?	Both the query and the name are converted with <code>toLowerCase()</code> before <code>contains()</code> .

9. Sorting, Comparator and lambdas

■ Required by the brief

In one sentence

Sorting orders a list using a Comparator - an object that says which of two items comes first.

service/RestaurantManager.java

```
public enum SortField {
    TIME("Time",
        Comparator.comparing(Reservation::getDateTime).thenComparing(Reservation::getId)),
    PARTY_SIZE("Party size", Comparator.comparingInt(Reservation::getPartySize)
        .thenComparing(Reservation::getDateTime)),
    CUSTOMER("Customer name", Comparator.comparing((Reservation r) -> r.getCustomer().getName(),
        String.CASE_INSENSITIVE_ORDER).thenComparing(Reservation::getDateTime)),
    TABLE("Table", Comparator.comparingInt((Reservation r) ->
        r.getTableNumber() == null ? Integer.MAX_VALUE : r.getTableNumber())
        .thenComparing(Reservation::getDateTime)),
    STATUS("Status",
        Comparator.comparing(Reservation::getStatus).thenComparing(Reservation::getDateTime));

    private final String label;
    private final Comparator<Reservation> comparator;

    SortField(String label, Comparator<Reservation> comparator) {
        this.label = label;
        this.comparator = comparator;
    }

    public Comparator<Reservation> comparator() {
        return comparator;
    }

    @Override
    public String toString() {
        return label;
    }
}
```

service/RestaurantManager.java

```
public static List<Reservation> sort(Collection<Reservation> input, SortField field, boolean
ascending) {
    List<Reservation> list = new ArrayList<>(input);
    Comparator<Reservation> comparator = field.comparator();
    Collections.sort(list, ascending ? comparator : comparator.reversed());
    return list;
}
```

Reading the comparator code

- `Reservation::getDateTime` is a *method reference* - shorthand for the lambda `r -> r.getDateTime()`.
- `Comparator.comparing(key)` builds a comparator that compares that key.
- `.thenComparing(...)` breaks ties - two parties of 4 are then ordered by time.
- `comparator.reversed()` gives descending order without writing a second comparator.
- `Collections.sort` uses TimSort: $O(n \log n)$ and *stable* (equal items keep their order).

Worked example

Before	Sorted by PARTY_SIZE ascending
Kavya (3, 20:00), Priya (2, 19:00), Diya (4, walk-in), Ananya (2, 21:00)	Priya (2, 19:00), Ananya (2, 21:00), Kavya (3), Diya (4) - the two parties of 2 are ordered by time (thenComparing)

Likely questions

Question	Short answer
Comparable vs Comparator?	Comparable is the one "natural" order inside the class (compareTo). Comparator is an external order; we need five different orders, so Comparators.
Why copy the list before sorting?	sort() works on its own copy so the caller's list (and the manager's master list) is never reordered.
What does the TreeMap have to do with sorting?	It keeps reservations permanently sorted by time, so "all in time order" needs no sort at all.

10. Exception handling

■ Required by the brief

In one sentence

An exception is an object thrown when something goes wrong; it travels up the call stack until a catch block handles it.

The hierarchy in this project

Class	Extends	Thrown when
RestaurantException	Exception (checked)	Base type for every business-rule failure
InvalidReservationException	RestaurantException	Bad name/phone/party size/time, illegal status change
InvalidTableException	RestaurantException	Bad table data or unsafe table change
TableNotAvailableException	RestaurantException	No suitable table free
NotFoundException	RestaurantException	Unknown reservation id, waitlist id or table

exception/RestaurantException.java

```
/** Base checked exception for every business-rule failure; the message is safe to show to the
host. */
public class RestaurantException extends Exception {
    public RestaurantException(String message) {
        super(message);
    }
}
```

How an error travels

1. The host types party size 21 and clicks Book.
2. ReservationsPanel.book() calls manager.createReservation(...).
3. createReservation calls Validator.validatePartySize(21), which throws InvalidReservationException("Party size cannot exceed 20...").
4. The exception passes up through createReservation (declared "throws RestaurantException").
5. book() catches it and shows the message in an error dialog. Nothing is saved.

gui/ReservationsPanel.java

```
private void book() {
    try {
        LocalDateTime when = TimeUtil.parse(dateTimeField.getText());
        Reservation r = manager.createReservation(nameField.getText(), phoneField.getText(),
            (Integer) partySpinner.getValue(), when, chosenSection(), chosenTable(),
            notesField.getText());
        GuiUtil.info(this, "Reservation " + r.getId() + " confirmed for " +
            r.getCustomer().getName() + "\n"
            + TimeUtil.formatDayTime(r.getDateTime()) + ", party of " + r.getPartySize()
            + ", table " + r.getTableNumber() + ".");
        clearForm();
    } catch (RestaurantException ex) {
        GuiUtil.error(this, ex.getMessage());
    }
}
```

Other exception techniques used

- try-with-resources in DataStore closes the file stream automatically even if writing fails.
- Catching specific types: TimeUtil catches DateTimeParseException and rethrows a friendly InvalidReservationException.
- Translating library exceptions: DataStore turns ClassNotFoundException / InvalidClassException into IOException with a clear message.
- Defensive catch in the monitor thread: a RuntimeException in one sweep is logged instead of killing the thread.

Viva traps - say this, not that

Checked exceptions (extends Exception) must be caught or declared - the compiler forces it. Unchecked (extends RuntimeException) do not. We chose checked so no GUI action can forget to handle a rule violation.

Do not say "the program crashes on bad input" - it never does; every action catches RestaurantException.

Likely questions

Question	Short answer
Why one base class RestaurantException?	The GUI can catch all business errors with one catch block, while code that cares can still catch a specific subtype.
What is the difference between throw and throws?	throw actually raises an exception object; throws in a method signature declares which checked exceptions it may raise.
What does finally do? Is it used?	It runs whether or not an exception occurred. ReservationsPanel.refresh() uses try/finally to always reset its "refreshing" flag.

11. Validation and regular expressions

■ Required by the brief

In one sentence

Validation checks input against the business rules before anything is changed, and explains exactly what is wrong.

Rule	Value	Message
Name	2-50 letters, spaces, . ' -	"Customer name must be 2-50 characters..."
Phone	10-13 digits, optional +	"Phone number must contain 10-13 digits..."
Party size	1-20	"Party size cannot exceed 20..."
Time	not in the past, 11:00-22:30, 15-minute slots, <= 60 days ahead	"...is in the past", "...outside service hours", "...15-minute slots"
Table capacity	1-20 seats; party must fit	"A party of 6 does not fit table 3 (4 seats)."
Date text	yyyy-MM-dd HH:mm, real calendar date	"\"2026-02-30 19:00\" is not a valid date/time..."

```
util/Validator.java
```

```
private static final Pattern NAME = Pattern.compile("[A-Za-z][A-Za-z .'-]{1,49}$");  
private static final Pattern PHONE = Pattern.compile("^\\+?\\d{10,13}$");
```

Reading the regular expressions

Part	Meaning
<code>^ ... \$</code>	The whole string must match (start to end).
<code>[A-Za-z]</code>	First character is a letter.
<code>[A-Za-z .'-]{1,49}</code>	Then 1 to 49 more letters, spaces, dots, apostrophes or hyphens - so "Asha O'Neil-Rao" is valid, "R2D2" is not.
<code>\\+?</code>	An optional plus sign.
<code>\\d{10,13}</code>	10 to 13 digits. Spaces, dashes and brackets are stripped first by <code>normalizePhone()</code> .

Strict date parsing: the formatter uses `uuuu` and `ResolverStyle.STRICT`, so 2026-02-30 is rejected instead of being quietly changed to 28 February.

util/TimeUtil.java

```
public static LocalDateTime parse(String text) throws InvalidReservationException {
    if (text == null || text.trim().isEmpty()) {
        throw new InvalidReservationException("Reservation date and time are required (" +
INPUT_PATTERN + ").");
    }
    try {
        return LocalDateTime.parse(text.trim(), INPUT);
    } catch (DateTimeParseException e) {
        throw new InvalidReservationException "\"" + text.trim() + "\" is not a valid date/time.
Use "
        + INPUT_PATTERN + ", e.g. 2026-10-02 19:30.");
    }
}
```

Why the spinners allow up to 99

The party-size and seat spinners in the GUI go up to 99, wider than the business limit of 20. If the spinner itself stopped at 20, typing 21 would silently snap back and the host would never learn why. Letting Validator reject it produces a clear message instead - and the rule stays in one place.

Likely questions

Question	Short answer
Where are the rules defined?	Only in Validator (constants like MAX_PARTY_SIZE = 20); the GUI, the manager and the sample data all call the same methods.
Why validate in the service layer and not only in the GUI?	So the rules hold no matter who calls the manager - GUI, sample data or a future web interface.

12. Threads: Runnable, Thread, synchronized and the Swing EDT

■ Required by the brief

In one sentence

A thread is an independent path of execution; using more than one lets the program wait or work in the background while the GUI stays responsive.

Required: Runnable / Thread to simulate waitlist notification. **Added:** a second monitor thread, synchronized methods, volatile, AtomicInteger and the Event Dispatch Thread rules that make threads safe with Swing.

12.1 Runnable and Thread

A `Runnable` is a task (it has one method, `run()`). A `Thread` is a worker that executes a task. When guests leave, the manager creates a new `Thread` for a `WaitlistNotifier` task:

service/RestaurantManager.java

```
private void scheduleAvailabilityCheck(Table table, long delayMs) {
    if (asyncNotifications) {
        Thread t = new Thread(new WaitlistNotifier(this, table.getNumber(), delayMs),
            "waitlist-notifier-table-" + table.getNumber());
        t.setDaemon(true);
        t.start();
    } else {
        offerTable(table);
    }
}
```

service/WaitlistNotifier.java

```
@Override
public void run() {
    try {
        if (bussingDelayMs > 0) {
            Thread.sleep(bussingDelayMs);
        }
        manager.processAvailability(tableNumber);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

start() vs run(): `start()` creates a new thread which then calls `run()`. Calling `run()` directly would execute it on the current thread and freeze the GUI for 2 seconds.

12.2 The monitor thread

service/ReservationMonitor.java

```
@Override
public void run() {
    while (running) {
        try {
            manager.runHousekeeping();
            Thread.sleep(intervalMs);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        } catch (RuntimeException e) {
            // Never let one bad sweep kill the monitor thread.
            System.err.println("Reservation monitor error: " + e);
        }
    }
}
```

- It is a *daemon* thread (`setDaemon(true)` in Main), so it does not keep the JVM alive after the window closes.
- `volatile boolean running` - writes by one thread are immediately visible to the others, so `stop()` works reliably.
- `InterruptedException` is handled by restoring the interrupt flag and exiting cleanly.

12.3 Timeline of one notification

Time	Thread	What happens
0 ms	Swing EDT	Host clicks Release Table -> <code>releaseTable(8)</code> marks the visit COMPLETED, table AVAILABLE, and starts the notifier thread. The click returns at once.
0-2000 ms	notifier	<code>Thread.sleep(2000)</code> - the table is being reset. The GUI stays usable.
2000 ms	notifier	<code>processAvailability(8)</code> (synchronized): first party in the <code>LinkedList</code> that fits -> <code>table.hold("W001")</code> .
2000 ms	notifier	Manager calls <code>listener.onNotification(ALERT, ...)</code> and <code>onDataChanged()</code> .
~2001 ms	Swing EDT	MainFrame's <code>invokeLater()</code> runnables run: [READY] line appended, banner turns yellow, tables repaint.

12.4 Why synchronized?

Three threads use the same data: the GUI, the monitor and notifier threads. Without protection, the monitor could hold table 4 for a booking at the exact moment the GUI seats a walk-in there - a *race condition* leaving the table both RESERVED and OCCUPIED. Every public `RestaurantManager` method is `synchronized`: only one thread at a time can be inside any of them, so each operation sees and leaves the data consistent.

12.5 Swing's single-thread rule

Swing components may only be touched from the Event Dispatch Thread (EDT). Background threads therefore never update the screen directly - `MainFrame` wraps every callback in `SwingUtilities.invokeLater(...)`, which queues the work on the EDT. Because the manager only queues work (it never waits for the GUI), the lock can never deadlock with the EDT.

Viva traps - say this, not that

`Thread.sleep()` does not release a lock - that is why the notifier sleeps before calling the synchronized `processAvailability()`, never inside it.

synchronized methods lock the object (this). `DataStore.save()` uses synchronized (manager) { ... } to take the same lock while writing the file.

Likely questions

Question	Short answer
Why a new thread for every released table instead of one shared thread?	Each release needs its own 2-second wait; separate short-lived threads keep them independent. A thread pool (ExecutorService) would be the next step at larger scale.
What is a daemon thread?	A background thread that does not stop the JVM from exiting when all normal threads have finished.
What would happen without invokeLater?	Swing could be updated from two threads at once, giving random painting glitches or exceptions; it is not thread-safe.
What is a race condition?	A bug where the result depends on the timing of two threads; prevented here with synchronized.

13. Swing GUI

■ Required by the brief

In one sentence

Swing is Java's built-in desktop GUI toolkit: windows, panels, tables, buttons and dialogs that react to events.

Component / class	Used for
JFrame (MainFrame)	The main window, menu bar, header, alert banner and notification log
JTabbedPane	The four tabs: Floor & Tables, Reservations, Walk-ins & Waitlist, Seating Reports
JTable + DefaultTableModel	The grids of tables, reservations and waiting parties (model holds rows; view displays them)
DefaultTableCellRenderer	StatusCellRenderer colours the status column
JTextField, JSpinner, JComboBox, JCheckBox	Form inputs
JOptionPane	Error, information and confirmation dialogs; the Add/Edit table dialog
BorderLayout, FlowLayout, GridBagLayout, GridLayout	Arranging components
javax.swing.Timer	Clock every 1 s, refresh every 30 s, autosave 1.5 s after a change
ActionListener, DocumentListener, ListSelectionListener, WindowAdapter	Reacting to clicks, typing in the search box, row selection, closing the window

Model-view separation in JTable

The JTable only draws; the DefaultTableModel holds the data. refresh() clears the model and refills it from the manager; the JTable repaints itself. Cells are made read-only by overriding isCellEditable:

gui/GuiUtil.java

```
static DefaultTableModel readOnlyModel(String... columns) {
    return new DefaultTableModel(columns, 0) {
        @Override
        public boolean isCellEditable(int row, int column) {
            return false;
        }
    };
}
```

Custom cell colours

gui/StatusCellRenderer.java

```
public Component getTableCellRendererComponent(JTable table, Object value, boolean isSelected,
                                                boolean hasFocus, int row, int column) {
    Component c = super.getTableCellRendererComponent(table, value, isSelected, hasFocus, row,
column);
    if (!isSelected) {
        c.setBackground(colorFor(value));
        c.setForeground(Color.BLACK);
    }
    return c;
}
```

Events

Every button is wired with a lambda, e.g. `GuiUtil.button("Book", e -> book())`. The lambda is an `ActionListener`: Swing calls it on the EDT when the button is clicked.

Likely questions

Question	Short answer
Why is <code>javax.swing.Timer</code> used and not <code>java.util.Timer</code> ?	<code>javax.swing.Timer</code> fires on the EDT, so its code may update components directly.
What is the EDT?	The Event Dispatch Thread - the single thread that processes all Swing events and painting.
How does the GUI learn about changes made by background threads?	Through <code>RestaurantListener</code> (Observer); <code>MainFrame</code> queues a refresh with <code>invokeLater</code> .

14. java.time and the Clock

□ Added by us - not in the brief

In one sentence

The java.time API (Java 8) represents dates and times as immutable, easy-to-compare objects.

- `LocalDateTime` - a date and time without a zone, e.g. 2026-10-04T19:30. Used for every booking.
- `Duration.between(a, b).toMinutes()` - minutes between two times (waiting time, lateness).
- `plusMinutes(90)`, `isBefore`, `isAfter` - the dining-window and hold-window arithmetic.
- `DateTimeFormatter` - parses "yyyy-MM-dd HH:mm" strictly and formats "Sun 04 Oct, 19:30".
- `Clock` - `RestaurantManager` reads "now" from a `Clock` passed to its constructor. Normally it is the system clock; for testing a fixed clock makes time-based rules (holds, lateness) repeatable.

Likely questions

Question	Short answer
Why <code>LocalDateTime</code> and not <code>java.util.Date</code> ?	It is immutable (thread-safe), has clear methods for adding minutes and comparing, and does not mix in time zones the restaurant does not need.
Why inject a <code>Clock</code> ?	So the logic does not call <code>LocalDateTime.now()</code> directly; a test can control time and check, for example, that a table is held exactly 30 minutes early.

15. Serialization and file I/O

□ Added by us - not in the brief

In one sentence

Serialization turns an object graph into bytes that can be written to a file and turned back into equal objects later.

- Model classes implement `Serializable` and declare `serialVersionUID = 1L` so the saved format stays compatible.
- Everything is saved as one `RestaurantSnapshot` (tables, reservations, waitlist, save time). The `HashMap` and `TreeMap` are not saved - they are rebuilt from the list on load, so they can never disagree with it.

persistence/DataStore.java

```
public static void save(RestaurantManager manager, File file) throws IOException {
    File dir = file.getAbsoluteFile().getParentFile();
    if (dir != null && !dir.exists() && !dir.mkdirs()) {
        throw new IOException("Could not create folder " + dir);
    }
    Path target = file.toPath();
    Path temp = new File(dir, file.getName() + ".tmp").toPath();
    // Lock the manager so background threads cannot change objects while they are being written.
    synchronized (manager) {
        try (ObjectOutputStream out = new ObjectOutputStream(
            new BufferedOutputStream(Files.newOutputStream(temp)))) {
            out.writeObject(manager.createSnapshot());
        }
    }
    try {
        Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING,
            StandardCopyOption.ATOMIC_MOVE);
    } catch (AtomicMoveNotSupportedException e) {
        Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING);
    }
}
```

Why write to a temporary file first?

If the computer crashes halfway through writing, the temporary file is damaged but the real data file is untouched. `Files.move(... ATOMIC_MOVE)` then replaces the old file in one step - you always have either the old data or the new data, never half of each.

Loading and recovery

- On start, Main loads `data/restaurant.dat` if it exists, otherwise loads sample data.
- A damaged file is renamed `restaurant.dat.corrupt-<time>` and the app starts with sample data instead of crashing.
- Visits left open from an earlier day are closed, past bookings that never arrived become no-shows, and yesterday's waitlist is cleared.

Likely questions

Question	Short answer
What is serialVersionUID for?	A version number for the class's saved form; if it differs between saving and loading, Java refuses to load (InvalidClassException) rather than mis-reading data.
What does transient mean? Did you use it?	A transient field is skipped when serializing. Not needed here - the listeners and indexes live in RestaurantManager, which is not serialized.
When is data saved?	1.5 s after any change (a Swing Timer coalesces bursts), on File > Save Now, and when the window closes.

16. Algorithms used in this project

◆ Design - how the brief's requirements are met

16.1 Best-fit table assignment

service/RestaurantManager.java

```
private Table bestFit(int partySize, LocalDateTime start, Section section, String ignoreId,
boolean mustBeFreeNow) {
    Table best = null;
    for (Table t : tables) {
        if (t.getCapacity() < partySize) continue;
        if (section != null && t.getSection() != section) continue;
        if (mustBeFreeNow && t.getStatus() != TableStatus.AVAILABLE) continue;
        if (findConflict(t, start, ignoreId) != null) continue;
        if (isBlockedByWaitlistHold(t, start)) continue;
        // tables are sorted by number, so ties keep the lowest-numbered table
        if (best == null || t.getCapacity() < best.getCapacity()) best = t;
    }
    return best;
}
```

Worked example

Table	Seats	Fits a party of 3?	Free 20:00-21:30?	Candidate
1	2	No	-	skip
3	4	Yes	No (booked 19:30)	skip
4	4	Yes	Yes	best so far (4 seats)
5	6	Yes	Yes	6 > 4, keep table 4

Result: table 4. Large tables stay free for large parties. If no table is found in the preferred section, the search is repeated in all sections.

16.2 Interval overlap (no double booking)

service/RestaurantManager.java

```
private Reservation findConflict(Table table, LocalDateTime start, String ignoreId) {
    LocalDateTime end = start.plusMinutes(Reservation.DINING_MINUTES);
    for (Reservation r : reservations) {
        if (!r.getStatus().isActive() || !isOnTable(r, table.getNumber()) ||
r.getId().equals(ignoreId)) continue;
        LocalDateTime busyFrom;
        LocalDateTime busyTo;
        if (r.getStatus() == ReservationStatus.SEATED) {
            // A seated party keeps the table at least until now, even if they are staying longer
            than planned.
            busyFrom = r.getSeatedAt();
            LocalDateTime planned = r.getSeatedAt().plusMinutes(Reservation.DINING_MINUTES);
            busyTo = planned.isAfter(now()) ? planned : now().plusMinutes(1);
        } else {
            busyFrom = r.getDateTime();
            busyTo = r.getEndTime();
        }
        if (start.isBefore(busyTo) && end.isAfter(busyFrom)) return r;
    }
    return null;
}
```

Existing booking	New booking	start < busyTo?	end > busyFrom?	Conflict?
19:00-20:30	20:00-21:30	20:00 < 20:30 yes	21:30 > 19:00 yes	Yes
19:00-20:30	20:30-22:00	20:30 < 20:30 no	-	No (back-to-back is fine)
19:00-20:30	17:30-19:00	17:30 < 20:30 yes	19:00 > 19:00 no	No

16.3 Offering a free table (priority order)

service/RestaurantManager.java

```
private boolean offerTable(Table table) {
    for (Reservation r : getReservationsBetween(now().minusMinutes(HOLD_WINDOW_MINUTES),
        now().plusMinutes(HOLD_WINDOW_MINUTES).plusSeconds(1))) {
        if (isOnTable(r, table.getNumber()) && holdIfDue(r)) return true;
    }
    // Don't hand the table to a walk-in if a booked party needs it within the next dining
    window.
    if (findConflict(table, now(), null) != null) return false;
    WaitlistEntry entry = waitlist.firstFitFor(table);
    if (entry == null) return false;
    entry.holdTable(table.getNumber(), now());
    table.hold(entry.getId());
    fire(NotificationType.ALERT, table + " is free - page waitlisted party " + entry
        + " (" + waitlist.positionOf(entry.getId()) + " in line, waited " +
        entry.minutesWaiting(now()) + " min).");
    return true;
}
```

1. A booking for this table due within 30 minutes gets it first.
2. If a booking needs the table within 90 minutes, it stays free (a walk-in would not finish in time).
3. Otherwise the first party in the LinkedList that fits gets it, and the host is told to page them.

Likely questions

Question	Short answer
Is best fit optimal?	It is a greedy rule that works well in practice and is easy to explain. A perfect assignment for a whole evening is a much harder optimisation problem (bin packing) and not needed for live front-desk decisions.
What is the time complexity of finding a table?	$O(T \times R)$: for each of T tables it checks R reservations for conflicts. With tens of tables and reservations that is instant.

17. Packages, compiling and running

□ Added by us - not in the brief

- Every class is in a package such as `com.restaurant.model`; the folder structure `src/com/restaurant/model/` matches it.
- `javac -d out -sourcepath src src/com/restaurant/Main.java` compiles `Main` and everything it uses into `out/`.
- `java -cp out com.restaurant.Main` starts the JVM with `out/` on the classpath and runs `main()`.
- No external libraries - only the JDK - so there is no build tool (Maven/Gradle) to set up.

Main.java

```
public static void main(String[] args) {
    if (GraphicsEnvironment.isHeadless()) {
        System.err.println("This application needs a graphical display (Swing GUI).");
        System.exit(1);
    }
    File dataFile = args.length > 0 ? new File(args[0]) : DataStore.DEFAULT_FILE;
    RestaurantManager manager = new RestaurantManager();
    String startupMessage = loadInitialData(manager, dataFile);

    SwingUtilities.invokeLater(() -> {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) {
            // fall back to the default look and feel
        }
        new MainFrame(manager, dataFile, startupMessage).setVisible(true);

        Thread monitor = new Thread(new ReservationMonitor(manager, MONITOR_INTERVAL_MS),
            "reservation-monitor");
        monitor.setDaemon(true);
        monitor.start();
    });
}
```

Likely questions

Question	Short answer
What is the classpath?	The list of folders/JARs where the JVM looks for .class files.
What does main() do here?	Checks there is a display, loads saved or sample data, opens MainFrame on the EDT and starts the monitor thread.
What is the difference between the JDK and the JRE?	The JRE runs Java programs; the JDK adds the compiler (javac) and tools needed to build them.